

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САМАРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИМЕНИ АКАДЕМИКА С.П. КОРОЛЕВА»

Лабораторная работа №5
Расширение возможностей классов табулированных функций:
переопределение методов Object

Выполнила:

Иванова Елизавета Вадимовна,
студентка группы 6301-030301D

Самара 2025

Задание 1

Реализованы все требуемые методы:

- `toString()` — возвращает строку вида `(x; y)`. Например: `(1.1; -7.5)`.
- `equals(Object o)` — корректно сравнивает две точки с учётом погрешности `double` (используется константа `EPS = 1e-10`). Возвращает `true`, только если оба объекта — точки с совпадающими координатами.
- `hashCode()` — реализован согласно условию: через `Double.doubleToLongBits()` и побитовое XOR. Это гарантирует, что точки с одинаковыми координатами имеют одинаковый хэш-код.
- `clone()` — возвращает новую точку с теми же координатами (глубокое копирование не требуется, так как нет ссылок на другие объекты).

Проверка подтвердила корректность: две одинаковые точки возвращают `equals = true` и одинаковый `hashCode`.

Задание 2

Реализованы все методы с учётом требований:

- `toString()` — выводит функцию в виде множества точек: `{(0.0; 1.2), (1.0; 3.8), (2.0; 15.2)}`.
- `equals(Object o)` — корректно сравнивает объекты. Метод проверяет, является ли аргумент экземпляром `TabulatedFunction`. Если да, он последовательно сравнивает все точки, вызывая `getPoint(i)` у обоих объектов и используя метод `equals()` из класса `FunctionPoint`. Этот подход гарантирует корректное сравнение чисел с плавающей точкой с учётом погрешности (`EPS = 1e-10`) и обеспечивает единообразие поведения для всех реализаций интерфейса.
- `hashCode()` — вычисляется на основе хэш-кодов всех точек и количества точек (чтобы функции разной длины не имели одинаковый хэш).
- `clone()` — реализует глубокое клонирование: создаёт новый массив и новые точки, чтобы копия была независима от оригинала.

При тестировании изменение оригинала не влияет на клон — клонирование работает корректно.

Задание 3

Аналогично реализованы методы:

- `toString()` — формат вывода идентичен `ArrayTabulatedFunction`.
- `equals(Object o)` сравнивает объекты через интерфейс `TabulatedFunction`. Для каждой точки списка вызывается метод `equals()` из класса `FunctionPoint`, который корректно сравнивает координаты с учётом погрешности (`EPS = 1e-10`). Это обеспечивает единообразие поведения при сравнении с любыми реализациями интерфейса.
- `hashCode()` — строится поочерёдно по точкам списка, с учётом количества точек.

- `clone()` — реализован через «пересборку» списка из массива точек. Это гарантирует независимость клона от оригинала без сложной реконфигурации узлов.

Особое внимание уделено инкапсуляции: метод `getPoint()` возвращает копию точки, а не ссылку на внутренний объект.

Задание 4

Интерфейс `TabulatedFunction` расширен:

- Добавлен метод `Object clone()`.
- Добавлен `extends Cloneable` — это требование JVM для корректной работы механизма клонирования.

Благодаря этому все реализации (`ArrayTabulatedFunction`, `LinkedListTabulatedFunction`) теперь являются клонируемыми с точки зрения JVM, и вызов `clone()` доступен через общий интерфейс.

Задание 5

Создан класс `Main`, в котором проведены все проверки:

- `toString()` — вывод корректен для обоих типов функций.
- `equals()` — проверены:

Сравнение одинаковых объектов → `true`.

Сравнение разных объектов → `false`.

Сравнение `ArrayTabulatedFunction` и `LinkedListTabulatedFunction` с одинаковыми точками → `true`.

- `hashCode()` — подтверждена согласованность с `equals`: одинаковые объекты имеют одинаковый хэш. При изменении даже одной координаты хэш меняется.
- `clone()` — после клонирования оригинал и клон изменялись независимо. Это подтверждает глубокое клонирование.

Все тесты пройдены успешно. Программа работает стабильно, ошибок нет.

```
toString
Array: {(0.0; 1.0), (1.0; 2.718), (2.0; 7.389)}
List:  {(0.0; 1.0), (1.0; 2.718), (2.0; 7.389)}
```

```
equals
array1.equals(array2): true
array1.equals(list1):  true
array1.equals(null):   false
```

```
hashCode
array1.hashCode(): 955510867
array2.hashCode(): 955510867
list1.hashCode():  955510867
array1.hashCode() == array2.hashCode(): true
array3.hashCode(): 1305746547
array1.equals(array3): false
```

```
clone
Клон создан: {(0.0; 1.0), (1.0; 2.718), (2.0; 7.389)}
После изменения оригинала:
Оригинал: 999.0
```

```
clone
Клон создан: {(0.0; 1.0), (1.0; 2.718), (2.0; 7.389)}
После изменения оригинала:
Оригинал: 999.0
Клон:      1.0
Список оригинал: 888.0
Список клон:      1.0
```

```
Process finished with exit code 0
```